

## Asynchronous JavaScript — নন-ব্লকিং মডেল ও অ্যাসিনক্রোনাস আর্কিটেকচার

JavaScript হলো একটি **Single-Threaded** ল্যাঙ্গুয়েজ, অর্থাৎ এটি একসাথে একটিমাত্র কাজ করতে পারে। কিন্তু বাস্তব অ্যাপ্লিকেশনে নেটওয়ার্ক রিকোয়েস্ট, সার্ভার থেকে ডাটা লোড, টাইমার বা ডেটাবেজ এক্সেসের মতো দীর্ঘস্থায়ী কাজের সময় ব্রাউজার যেন জমে (Freeze) না যায়, সেজন্য জাভাস্ক্রিপ্ট **Non-blocking Asynchronous** মডেল ব্যবহার করে।

Call Stack (JS Engine) → Web APIs (Timer/Network) → Callback / Microtask Queue → Event Loop

React, Next.js, Node.js কিংবা Express-এ API Integration ও Backend Communication-এর মূল চালিকাশক্তিই হলো Asynchronous JavaScript! 🔥

## Part 01 — Asynchronous Foundations, Browser Runtime, Call Stack & Event Loop (21.1 – 21.10)

### 21.1 Synchronous vs Asynchronous

#### ♦ Synchronous JavaScript

Synchronous মানে:

একটি কাজ শেষ হওয়ার পর পরের কাজ শুরু হবে।

#### 💡 Example

21.1 JAVASCRIPT ASYNC

ES2026

```
console.log("Task 1");  
console.log("Task 2");  
console.log("Task 3");
```

```
Task 1  
Task 2  
Task 3
```

এখানে JavaScript:

```
Task 1 → শেষ  
↓  
Task 2 → শেষ  
↓  
Task 3 → শেষ
```

#### 💡 ♦ Real-Life Example

ধরো তুমি restaurant-এ গেলে।

একজন waiter:

- Customer 1-এর order নিল
- Customer 1-এর খাবার তৈরি হওয়া পর্যন্ত দাঁড়িয়ে থাকল
- খাবার দিল
- তারপর Customer 2-এর order নিল

এটা inefficient!

এটাই synchronous-এর মতো।

## 21.2 Asynchronous JavaScript

Asynchronous মানে:

একটি কাজের জন্য অপেক্ষা করতে গিয়ে JavaScript পুরো program-কে আটকে রাখে না।

### 💡 Example

21.2 JAVASCRIPT ASYNC ES2026

```
console.log("Start");

setTimeout(() => {
  console.log("Data received");
}, 2000);

console.log("End");
```

#### CONSOLE OUTPUT

```
Start
End
Data received
```

এখানে কী হলো?

```
Start
↓
setTimeout শুরু
↓
JavaScript অপেক্ষা না করে
↓
End
↓
2 seconds পরে
↓
Data received
```

## 21.3 setTimeout()

setTimeout() নির্দিষ্ট সময় পরে একটি function execute করে।

### ⚙️ Syntax

21.3 JAVASCRIPT ASYNC ES2026

```
setTimeout(function, delay);
```

### 💡 Example

21.3 JAVASCRIPT ASYNC ES2026

```
setTimeout(() => {
  console.log("Hello");
}, 3000);
```

৩ সেকেন্ড পরে:

#### CONSOLE OUTPUT

```
Hello
```

### 💡 Real-Life Example

তুমি বললে:

"আমাকে 5 মিনিট পরে মনে করিয়ে দিও।"

JavaScript-এর ক্ষেত্রে:

21.3 JAVASCRIPT ASYNC ES2026

```
setTimeout(() => {
  console.log("Reminder!");
}, 5000);
```

#### 21.4 setTimeout() কি JavaScript-এর অংশ?

এখানে একটি গুরুত্বপূর্ণ বিষয় আছে।

`setTimeout()` আসলে JavaScript language-এর core feature নয়।

এটি environment থেকে আসে।

Browser-এ:

```
JavaScript
↓
Browser Web APIs
↓
setTimeout()
```

Node.js-এ Node-এর runtime timer APIs provide করে।

#### 21.5 JavaScript কেন Async হতে পারে?

JavaScript-এর মূল execution model হলো **single-threaded**।

অর্থাৎ সাধারণভাবে একটি JavaScript execution thread এক সময়ে একটি কাজ execute করে।

কিন্তু Browser environment বিভিন্ন কাজ JavaScript-এর বাইরে handle করতে পারে।

যেমন:

```
JavaScript
↓
Web API
↓
Timer / Network / DOM
↓
Task Queue
↓
Event Loop
↓
JavaScript
```

এই mechanism-টাই asynchronous JavaScript বোঝার জন্য খুব গুরুত্বপূর্ণ।

#### 21.6 Call Stack

**Call Stack** হলো JavaScript-এর execution-এর জায়গা।

Example:

```
21.6 JAVASCRIPT ASYNC ES2026

function one() {
  console.log("One");
}

function two() {
  one();
  console.log("Two");
}

two();
```

Execution:

```
two()
↓
one()
↓
console.log()
```

Call Stack ধারণাগতভাবে:

|             |
|-------------|
| console.log |
| one()       |
| two()       |

যে function আগে আসে, তার execution complete হয়ে stack থেকে বের হয়।

## 21.7 Event Loop

এখন আসল magic:

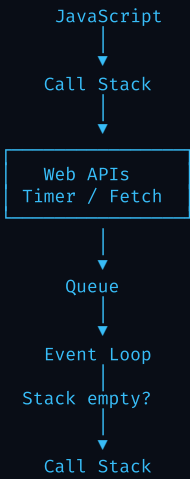
### Event Loop কী?

Event Loop check করে:

"Call Stack কি empty?"

যদি empty হয় এবং queue-তে pending কাজ থাকে, তাহলে appropriate callback execution-এর জন্য stack-এ পার্তানো হয়।

Conceptually:



## 21.8 Callback Function

Callback হলো:

একটি function-কে অন্য function-এর argument হিসেবে দেওয়া, যাতে পরে সেটি execute করা যায়।

### Example

21.8 JAVASCRIPT ASYNC

ES2026

```
function greet(name) {  
  console.log("Hello " + name);  
}  
  
function processUser(callback) {  
  callback("Shariar");  
}  
  
processUser(greet);
```

CONSOLE OUTPUT

Hello Shariar

এখানে:

21.8 JAVASCRIPT ASYNC

ES2026

greet

হলো callback function।

## 21.9 Asynchronous Callback

21.9 JAVASCRIPT ASYNC

ES2026

```
console.log("Start");

setTimeout(() => {
  console.log("Hello");
}, 2000);

console.log("End");
```

OUTPUT / TEXT

```
Start
End
Hello
```

`setTimeout()` -এর ভিতরের function হলো asynchronous callback।

## 21.10 Callback Hell

একাধিক asynchronous কাজ একটার result-এর উপর আরেকটা নির্ভর করলে code nested হয়ে যেতে পারে।

### 💡 Example

21.10 JAVASCRIPT ASYNC

ES2026

```
getUser(function(user) {
  getPosts(user, function(posts) {
    getComments(posts, function(comments) {
      getLikes(comments, function(likes) {
        console.log(likes);
      });
    });
  });
});
```

এটাকে দেখতে অনেকটা:

```
getUser()
  ↓
getPosts()
  ↓
getComments()
  ↓
getLikes()
```

এটাকে বলা হয়:

# 🔥 Callback Hell

### ★ কেন সমস্যা?

- Code difficult to read
- Debugging কঠিন
- Error handling complicated
- Maintenance কঠিন
- Nested code বাড়াতে থাকে

এই সমস্যা solve করার জন্য এসেছে:

# ★ Promise

### 21.11 Promise

Promise হলো:

ভবিষ্যতে কোনো asynchronous operation-এর result পাওয়া যাবে—এই promise/প্রতিশ্রুতি।

Real-life example:

তুমি online-এ product order করলে।

Order করার পর:

```
Order placed
  ↓
Processing
  ↓
Shipped
  ↓
Delivered
```

তুমি জানো result এখনো আসেনি, কিন্তু পরে আসবে।

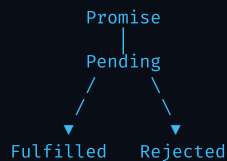
এটাই Promise-এর ধারণার মতো।

### 21.12 Promise-এর ৩টি State

একটি Promise-এর প্রধান ৩টি state:

| State     | Meaning           |
|-----------|-------------------|
| pending   | কাজ চলছে          |
| fulfilled | কাজ সফল হয়েছে    |
| rejected  | কাজ ব্যর্থ হয়েছে |

Diagram:



### 21.13 Promise তৈরি করা

#### ⚙ Syntax

21.13 JAVASCRIPT ASYNC

ES2026

```
const promise = new Promise((resolve, reject) => {
});
```

এখানে:

21.13 JAVASCRIPT ASYNC

ES2026

```
resolve()
```

মানে success।

21.13 JAVASCRIPT ASYNC

ES2026

```
reject()
```

মানে failure।

### 21.14 Simple Promise Example

21.14 JAVASCRIPT ASYNC

ES2026

```
const promise = new Promise((resolve, reject) => {  
  let success = true;  
  
  if (success) {  
    resolve("Operation successful!");  
  } else {  
    reject("Operation failed!");  
  }  
  
});
```

Promise এখন:

OUTPUT / TEXT

fulfilled

কারণ:

21.14 JAVASCRIPT ASYNC

ES2026

```
success = true
```

### 21.15 Promise-এর Result নেওয়া — .then()

21.15 JAVASCRIPT ASYNC

ES2026

```
const promise = new Promise((resolve, reject) => {  
  resolve("Data loaded!");  
});  
  
promise.then((result) => {  
  console.log(result);  
});
```

CONSOLE OUTPUT

Data loaded!

.then() সাধারণত successful result handle করে।

### 21.16 .catch()

যদি Promise reject হয়:

21.16 JAVASCRIPT ASYNC

ES2026

```
const promise = new Promise((resolve, reject) => {  
  reject("Something went wrong!");  
});  
  
promise  
  .then((result) => {  
    console.log(result);  
  })  
  .catch((error) => {  
    console.log(error);  
  });
```

CONSOLE OUTPUT

Something went wrong!

### 21.17 .finally()

.finally() success বা failure—দুই ক্ষেত্রেই execute হয়।

21.17 JAVASCRIPT ASYNC

ES2026

```
const promise = new Promise((resolve) => {
  resolve("Success");
});

promise
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log(error);
  })
  .finally(() => {
    console.log("Finished");
  });
```

#### CONSOLE OUTPUT

```
Success
Finished
```

### 21.18 Promise দিয়ে Delay তৈরি

21.18 JAVASCRIPT ASYNC

ES2026

```
function delay(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

delay(2000).then(() => {
  console.log("2 seconds completed");
});
```

২ seconds পরে:

#### CONSOLE OUTPUT

```
2 seconds completed
```

### 21.19 Promise Chaining

একটি Promise-এর result দিয়ে পরের কাজ করা যায়।

21.19 JAVASCRIPT ASYNC

ES2026

```
Promise.resolve(10)
  .then((number) => {
    return number * 2;
  })
  .then((number) => {
    return number + 5;
  })
  .then((result) => {
    console.log(result);
  });
```

#### CONSOLE OUTPUT

```
25
```

কারণ:

```
10
↓ ×2
20
↓ +5
25
```

## 21.20 Promise Chain কেন গুরুত্বপূর্ণ?

Real API application:

```
Login
↓
Get User
↓
Get Profile
↓
Get Orders
↓
Show Dashboard
```

Promise chaining দিয়ে এগুলো manage করা যায়।

## 21.21 Promise Error Handling

21.21 JAVASCRIPT ASYNC

ES2026

```
Promise.reject("Server Error")
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.error("Error:", error);
  });
```

CONSOLE OUTPUT

Error: Server Error

## 21.22 Promise.resolve()

Already successful Promise তৈরি করতে:

21.22 JAVASCRIPT ASYNC

ES2026

```
const promise = Promise.resolve("Hello");

promise.then((value) => {
  console.log(value);
});
```

OUTPUT / TEXT

Hello

## 21.23 Promise.reject()

21.23 JAVASCRIPT ASYNC

ES2026

```
const promise = Promise.reject("Failed");

promise.catch((error) => {
  console.log(error);
});
```

OUTPUT / TEXT

Failed

**21.24** Promise.all()

একসাথে একাধিক Promise চালাতে খুব usefull

ধরো:

OUTPUT / TEXT

```
User data
Product data
Order data
```

তিনটিই দরকার।

21.24 JAVASCRIPT ASYNC

ES2026

```
const userPromise = Promise.resolve("User");
const productPromise = Promise.resolve("Products");
const orderPromise = Promise.resolve("Orders");

Promise.all([
  userPromise,
  productPromise,
  orderPromise
])
.then((results) => {
  console.log(results);
});
```

CONSOLE OUTPUT

```
["User", "Products", "Orders"]
```

**💡 Real-Life Example**

Dashboard load করতে:

OUTPUT / TEXT

```
GET /user
GET /products
GET /orders
```

সবগুলো successful হলে:

21.24 JAVASCRIPT ASYNC

ES2026

```
Promise.all([
  getUser(),
  getProducts(),
  getOrders()
])
```

তারপর dashboard render করা যায়।

**21.25** Promise.all() -এর গুরুত্বপূর্ণবিষয়

যদি একটি Promise reject করে:

21.25 JAVASCRIPT ASYNC

ES2026

```
Promise.all([
  Promise.resolve("A"),
  Promise.reject("Error"),
  Promise.resolve("C")
])
.catch((error) => {
  console.log(error);
});
```

OUTPUT / TEXT

```
Error
```

অর্থাৎ Promise.all() সাধারণভাবে সবগুলো সফল না হলে reject করে।

### 21.26 Promise.allSettled()

সব Promise-এর final result জানতে চাইলে:

21.26 JAVASCRIPT ASYNC

ES2026

```
Promise.allSettled([
  Promise.resolve("Success"),
  Promise.reject("Failed")
])
.then((results) => {
  console.log(results);
});
```

Result conceptually:

21.26 JAVASCRIPT ASYNC

ES2026

```
[
  {
    status: "fulfilled",
    value: "Success"
  },
  {
    status: "rejected",
    reason: "Failed"
  }
]
```

এখানে একটি fail করলেও অন্য result পাওয়া যায়।

### 21.27 Promise.race()

যে Promise আগে settle করবে, তার result পাওয়া যাবে।

21.27 JAVASCRIPT ASYNC

ES2026

```
const first = new Promise((resolve) => {
  setTimeout(() => resolve("First"), 1000);
});

const second = new Promise((resolve) => {
  setTimeout(() => resolve("Second"), 2000);
});

Promise.race([first, second])
  .then((result) => {
    console.log(result);
  });
```

CONSOLE OUTPUT

First

কারণ `first` আগে complete করেছে।

### 21.28 Promise.any()

যে Promise প্রথম **fulfilled** হবে তার result দেয়।

21.28 JAVASCRIPT ASYNC

ES2026

```
Promise.any([
  Promise.reject("Failed 1"),
  Promise.resolve("Success"),
  Promise.resolve("Success 2")
])
.then((result) => {
  console.log(result);
});
```

OUTPUT / TEXT

Success

**21.29** `async`` Function

Promise handling আরও সহজ করার জন্য:

#  `async``

একটি function-এর সামনে `async`` দিলে function সবসময় Promise return করে।

21.29 JAVASCRIPT ASYNC

ES2026

```
async function greet() {
  return "Hello";
}
```

এটা conceptually:

21.29 JAVASCRIPT ASYNC

ES2026

```
Promise.resolve("Hello")
```

এর মতো।

**21.30** `async`` Example

21.30 JAVASCRIPT ASYNC

ES2026

```
async function getMessage() {
  return "Hello Shariar";
}

getMessage().then((message) => {
  console.log(message);
});
```

OUTPUT / TEXT

Hello Shariar

**21.31** `await``

`await`` Promise-এর result পাওয়ার জন্য অপেক্ষা করার মতো syntax দেয়।

21.31 JAVASCRIPT ASYNC

ES2026

```
function getData() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve("Data received");
    }, 2000);
  });
}

async function showData() {
  const data = await getData();

  console.log(data);
}

showData();
```

২ seconds পরে:

CONSOLE OUTPUT

Data received

### 21.32 `async/await` কেন এত জনপ্রিয়?

আগে:

21.32 JAVASCRIPT ASYNC

ES2026

```
getData()
  .then((data) => {
    return processData(data);
  })
  .then((result) => {
    console.log(result);
  });
```

`async/await`:

21.32 JAVASCRIPT ASYNC

ES2026

```
async function run() {

  const data = await getData();

  const result = await processData(data);

  console.log(result);
}
```

দ্বিতীয়টি অনেকটা synchronous code-এর মতো readable।

### 21.33 `await` কি পুরো JavaScript বন্ধ করে দেয়?

✗ না।

এটি খুব গুরুত্বপূর্ণ।

21.33 JAVASCRIPT ASYNC

ES2026

```
async function test() {

  console.log("A");

  await delay(2000);

  console.log("B");
}

test();

console.log("C");
```

OUTPUT / TEXT

A  
C  
B

`await` asynchronous function-এর ওই অংশের execution pause করে, কিন্তু পুরো JavaScript environment/blocking করে না।

## 21.34 async/await` Error Handling

await -এর সাথে সাধারণত:

21.34 JAVASCRIPT ASYNC

ES2026

try/catch

ব্যবহার করা হয়।

21.34 JAVASCRIPT ASYNC

ES2026

```
function getData() {
  return Promise.reject("Server Error");
}

async function loadData() {
  try {
    const data = await getData();
    console.log(data);
  } catch (error) {
    console.log("Error:", error);
  }
}

loadData();
```

CONSOLE OUTPUT

Error: Server Error

## 21.35 try/catch/finally` + async

21.35 JAVASCRIPT ASYNC

ES2026

```
async function loadData() {
  try {
    const data = await getData();
    console.log(data);
  } catch (error) {
    console.log("Error:", error);
  } finally {
    console.log("Request finished");
  }
}
```

### 21.36 Sequential Async Operations

ধরো:

```
Login
↓
Get profile
↓
Get orders
```

একটির result ছাড়া পরেরটি চলবে না।

21.36 JAVASCRIPT ASYNC

ES2026

```
async function run() {

  const user = await login();

  const profile = await getProfile(user);

  const orders = await getOrders(profile);

  console.log(orders);
}
```

এটা sequential।

### 21.37 Parallel Async Operations

যদি কাজগুলো একে অপরের উপর depend না করে:

21.37 JAVASCRIPT ASYNC

ES2026

```
async function loadDashboard() {

  const [users, products, orders] = await Promise.all([
    getUsers(),
    getProducts(),
    getOrders()
  ]);

  console.log(users);
  console.log(products);
  console.log(orders);
}
```

এখানে তিনটি operation parallelভাবে শুরু হয়।

🚀 Important

এটা:

21.37 JAVASCRIPT ASYNC

ES2026

```
const users = await getUsers();
const products = await getProducts();
const orders = await getOrders();
```

এর চেয়ে independent operations-এর ক্ষেত্রে বেশি efficient হতে পারে।

## 21.38 await` Inside Loop

Example:

21.38 JAVASCRIPT ASYNC

ES2026

```
async function process() {  
  for (const id of [1, 2, 3]) {  
    const data = await getData(id);  
    console.log(data);  
  }  
}
```

এখানে:

```
1 → complete  
↓  
2 → complete  
↓  
3 → complete
```

Sequential execution।

যদি independent হয়, তাহলে অনেক সময়:

21.38 JAVASCRIPT ASYNC

ES2026

```
const results = await Promise.all(  
  [1, 2, 3].map(id => getData(id))  
);
```

better approach হতে পারে।

## Part 05 — Engine Internals, Microtasks, Fetch API & Production Architecture (21.39 – 21.55)

### 21.39 Microtask vs Macrotask

এখন একটু advanced concept।

JavaScript asynchronous execution-এ বিভিন্ন queue থাকে।

**সাধারণভাবে:**

#### Microtask Queue

- Promise `.then()`
- `.catch()`
- `.finally()`
- `queueMicrotask()`

#### Task/Macrotask Queue

- `setTimeout`
- `setInterval`
- কিছু browser events

## 21.40 Event Loop Example

21.40 JAVASCRIPT ASYNC

ES2026

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

console.log("4");
```

CONSOLE OUTPUT

```
1
4
3
2
```

কেন?

প্রথমে synchronous:

OUTPUT / TEXT

```
1
4
```

তারপর microtask:

OUTPUT / TEXT

```
3
```

তারপর task:

OUTPUT / TEXT

```
2
```

অর্থাৎ conceptually:

```
Synchronous code
↓
Microtasks
↓
Tasks
```

## 21.41 setTimeout(..., 0) 'মানে কি হচ্ছে হচ্ছে?

❌ না।

21.41 JAVASCRIPT ASYNC

ES2026

```
setTimeout(() => {
  console.log("Hello");
}, 0);
```

এর অর্থ:

callback-টি যত দ্রুত সম্ভব পরে task queue-তে handle করার সুযোগ পাবে।

এটি synchronous code-এর আগে execute হয় না।

## 21.42 Fetch API

এখন সবচেয়ে গুরুত্বপূর্ণ practical বিষয়:

# 🌐 Fetch API

Frontend developer হিসেবে server/API থেকে data আনতে `fetch()` খুব গুরুত্বপূর্ণ।

### 💡 Example

21.42 JAVASCRIPT ASYNC

ES2026

```
fetch("https://example.com")
  .then(response => {
    console.log(response);
  });
```

`fetch()` একটি Promise return করে।

## 21.43 Fetch + JSON

API থেকে JSON data আনতে:

21.43 JAVASCRIPT ASYNC

ES2026

```
fetch("https://api.example.com/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

Flow:

```
fetch()
↓
Response
↓
response.json()
↓
JSON data
↓
Use data
```

## 21.44 Fetch with `async/await`

এটা আরও readable:

21.44 JAVASCRIPT ASYNC

ES2026

```
async function getUsers() {
  try {
    const response = await fetch(
      "https://api.example.com/users"
    );
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}

getUsers();
```

## 21.45 HTTP Error সম্পর্কে গুরুত্বপূর্ণ বিষয়

`fetch()` সাধারণত HTTP 404 বা 500 পেলেই Promise reject করে না।

তাই:

21.45 JAVASCRIPT ASYNC

ES2026

```
const response = await fetch(url);
```

এরপর check করা ভালো:

21.45 JAVASCRIPT ASYNC

ES2026

```
if (!response.ok) {  
  throw new Error(`HTTP Error: ${response.status}`);  
}
```

Complete:

21.45 JAVASCRIPT ASYNC

ES2026

```
async function getUsers() {  
  
  try {  
  
    const response = await fetch(  
      "https://example.com/users"  
    );  
  
    if (!response.ok) {  
      throw new Error(  
        `HTTP Error: ${response.status}`  
      );  
    }  
  
    const data = await response.json();  
  
    console.log(data);  
  
  } catch (error) {  
    console.error(error);  
  }  
}
```

## 21.46 Real-Life Login Example

ধরো frontend থেকে login request:

21.46 JAVASCRIPT ASYNC

ES2026

```
async function login(email, password) {  
  
  try {  
  
    const response = await fetch("/api/login", {  
  
      method: "POST",  
  
      headers: {  
        "Content-Type": "application/json"  
      },  
  
      body: JSON.stringify({  
        email,  
        password  
      })  
  
    });  
  
    if (!response.ok) {  
      throw new Error("Login failed");  
    }  
  
    const data = await response.json();  
  
    console.log("Login successful:", data);  
  
  } catch (error) {  
  
    console.error("Login error:", error);  
  
  }  
  
}
```

এখানে Chapter 19-এর JSON + Chapter 20-এর modules + Chapter 21-এর async concept একসাথে ব্যবহার হচ্ছে।

## 21.47 Async JavaScript-এর Complete Flow

একটি modern web application-এ:

```
User clicks button  
↓  
Event Handler  
↓  
async function  
↓  
fetch()  
↓  
Promise  
↓  
Server/API  
↓  
Response  
↓  
await  
↓  
JSON  
↓  
Data processing  
↓  
DOM update
```

এটাই frontend development-এর খুব common workflow।

## 21.48 Callback vs Promise vs Async/Await

| Feature        | Callback             | Promise               | Async/Await            |
|----------------|----------------------|-----------------------|------------------------|
| Readability    | কম                   | ভালো                  | ★ খুব ভালো             |
| Nested code    | বেশি হতে পারে        | কম                    | কম                     |
| Error handling | complicated হতে পারে | <code>.catch()</code> | <code>try/catch</code> |
| Chaining       | কঠিন                 | <code>.then()</code>  | সহজ                    |
| Modern JS      | পুরোনো/common        | Standard              | ★ খুব common           |

## 21.49 Real-Life Mini Project — Fake API

21.49 JAVASCRIPT ASYNC

ES2026

```
function getUser() {  
  return new Promise((resolve) => {  
    setTimeout(() => {  
      resolve({  
        id: 1,  
        name: "Shariar",  
        role: "Full Stack Developer"  
      });  
    }, 2000);  
  });  
}  
  
async function showUser() {  
  console.log("Loading ...");  
  
  try {  
    const user = await getUser();  
    console.log("User:", user);  
  } catch (error) {  
    console.error(error);  
  }  
}  
  
showUser();
```

প্রথমে:

CONSOLE OUTPUT

Loading ...

২ seconds পরে:

OUTPUT / TEXT

```
User: {  
  id: 1,  
  name: "Shariar",  
  role: "Full Stack Developer"  
}
```

## 21.50 Async Function + Loading UI

Real website-৯:

21.50 JAVASCRIPT ASYNC

ES2026

```
async function loadProducts() {  
  
    const loading = document.querySelector("#loading");  
  
    try {  
  
        loading.textContent = "Loading ... ";  
  
        const response = await fetch("/api/products");  
  
        if (!response.ok) {  
            throw new Error("Failed to load products");  
        }  
  
        const products = await response.json();  
  
        console.log(products);  
  
    } catch (error) {  
  
        console.error(error);  
  
    } finally {  
  
        loading.textContent = "";  
  
    }  
}
```

এখানে:

```
Start  
↓  
Loading ...  
↓  
API request  
↓  
Success/Error  
↓  
finally  
↓  
Loading remove
```

## 21.51 Promise-এর কিছু গুরুত্বপূর্ণ ভুল

✗ ভুল

21.51 JAVASCRIPT ASYNC

ES2026

```
const result = getData();  
  
console.log(result);
```

যদি `getData()` Promise return করে:

OUTPUT / TEXT

```
Promise { <pending> }
```

পেতে পারো।

✓ সঠিক

21.51 JAVASCRIPT ASYNC

ES2026

```
const result = await getData();
```

অথবা:

21.51 JAVASCRIPT ASYNC

ES2026

```
getData().then(result => {  
    console.log(result);  
});
```

## 21.52 `await` কোথায় ব্যবহার করা যায়?

সাধারণভাবে `await` ব্যবহার করা যায়:

### `async function` -এর ভিতরে

21.52 JAVASCRIPT ASYNC

ES2026

```
async function test() {  
    const data = await getData();  
}
```

এছাড়া modern JavaScript modules-এ **top-level** `await` -ও ব্যবহার করা যায়।

যেমন module:

21.52 JAVASCRIPT ASYNC

ES2026

```
const data = await getData();  
  
console.log(data);
```

## 21.53 Promise Constructor-এর Common Mistake

### ✗ অপ্রয়োজনীয়ভাবে Promise wrap করা

21.53 JAVASCRIPT ASYNC

ES2026

```
return new Promise((resolve, reject) => {  
    fetch(url)  
        .then(response => resolve(response))  
        .catch(error => reject(error));  
});
```

অনেক ক্ষেত্রে এটি দরকার হয় না।

সরাসরি:

21.53 JAVASCRIPT ASYNC

ES2026

```
return fetch(url);
```

ব্যবহার করা যায়।

## 21.54 Promise-এর Mental Model

মনে রাখবে:

```
Promise  
├── Pending  
├── Fulfilled  
└── Rejected
```

তারপর:

```
.then()    → Success  
.catch()   → Error  
.finally() → Always
```

## 21.55 Async/Await Mental Model

মনে রাখবে:

21.55 JAVASCRIPT ASYNC

ES2026

```
async function load() {  
  
  try {  
  
    const data = await getData();  
  
  } catch (error) {  
  
    console.log(error);  
  
  }  
  
}
```

মানে:

```
async  
↓  
Promise-based function  
↓  
await result  
↓  
try/catch error
```

**MUST KNOW** 🔥 21.56 Must Know — Asynchronous JavaScript Core Pillars

Chapter 21 থেকে অবশ্যই জানতে হবে:

**Basic**

- Synchronous
- Asynchronous
- `setTimeout()`
- Callback
- Call Stack
- Event Loop

**Promise**

- Promise
- `pending`
- `fulfilled`
- `rejected`
- `resolve()`
- `reject()`
- `.then()`
- `.catch()`
- `.finally()`
- Promise chaining

**Promise Methods**

JAVASCRIPT ASYNC

ES2026

```
Promise.resolve()
Promise.reject()
Promise.all()
Promise.allSettled()
Promise.race()
Promise.any()
```

**Modern Async**

- `async`
- `await`
- `try/catch`
- Sequential async
- Parallel async
- `Promise.all()`

**Practical**

- Fetch API
- JSON response
- HTTP error checking
- API error handling
- Loading state

**Advanced**

- Microtask
- Task/Macrotask
- Event Loop execution order

JAVASCRIPT ASYNC

ES2026

```
// Callback
setTimeout(() => {
  console.log("Hello");
}, 1000);
```

JAVASCRIPT ASYNC

ES2026

```
// Promise
const p = new Promise((resolve, reject) => {
  resolve("Success");
});
```

JAVASCRIPT ASYNC

ES2026

```
// then
p.then(data => {
  console.log(data);
});
```

JAVASCRIPT ASYNC

ES2026

```
// catch
p.catch(error => {
  console.log(error);
});
```

JAVASCRIPT ASYNC

ES2026

```
// async
async function test() {
}
```

JAVASCRIPT ASYNC

ES2026

```
// await
const data = await getData();
```

JAVASCRIPT ASYNC

ES2026

```
// Promise.all
const results = await Promise.all([
  getUsers(),
  getProducts()
]);
```

JAVASCRIPT ASYNC

ES2026

```
// Fetch
const response = await fetch(url);
```

JAVASCRIPT ASYNC

ES2026

```
// JSON
const data = await response.json();
```

JAVASCRIPT ASYNC

ES2026

```
// Error
if (!response.ok) {
  throw new Error("Request failed");
}
```



### Practice 1

একটি function বানাও যেটি 3 seconds পরে:

OUTPUT / TEXT

Hello JavaScript

print করবে।

### Practice 2

একটি Promise তৈরি করো:

OUTPUT / TEXT

Success

return করবে।

তারপর `.then()` দিয়ে output দেখাও।

### Practice 3

একটি Promise তৈরি করো যেটা reject করবে:

OUTPUT / TEXT

Network Error

তারপর `.catch()` দিয়ে error দেখাও।

### Practice 4

`async/await` দিয়ে একটি fake API function তৈরি করো যেটি 2 seconds পরে:

JAVASCRIPT ASYNC

ES2026

```
{
  name: "Shariar",
  age: 24
}
```

return করবে।

### Practice 5

এই code-এর output predict করো:

JAVASCRIPT ASYNC

ES2026

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

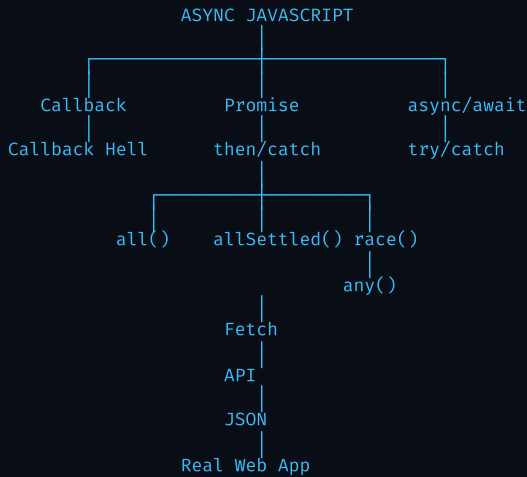
console.log("D");
```

Answer:

OUTPUT / TEXT

A  
D  
C  
B

পুরো Chapter-টা এভাবে মনে রাখো:



✦ সবচেয়ে গুরুত্বপূর্ণ concept:

JavaScript single-threaded হলেও asynchronous programming-এর মাধ্যমে timer, network request, API call ইত্যাদি কাজ handle করতে পারে।

আর modern web development-এ তোমার সবচেয়ে বেশি ব্যবহার হবে:

JAVASCRIPT ASYNC

ES2026

```

async function getData() {
  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error("Request failed");
    }

    const data = await response.json();

    return data;
  } catch (error) {
    console.error(error);
  }
}
  
```

এই pattern তুমি **React + Node.js + Express + REST API + Full Stack Development**-এ বারবার দেখতে পাবে।

পরবর্তী অধ্যায়ে আমরা শিখব **HTTP Protocols, REST APIs, Request Headers, Status Codes (200, 201, 400, 401, 404, 500)** এবং ব্রাউজারের আধুনিক `fetch()` API দিয়ে ফুল-স্ট্যাক নেটওয়ার্ক ইন্টারঅ্যাকশন ও এরর হ্যান্ডলিং কৌশল।